Caminho mínimo

Prof. Eduardo Enari

Objetivo

- Apresentar os fundamentos do problema do caminho mínimo em grafos e algoritmos que foram formulados para resolver esse problema.

Problema do caminho mínimo

- Uma das aplicações mais comuns é utilizar grafos para modelar mapas.
  - Vértices podem representar cidades, esquinas ou pontos de referência etc.
  - Arestas podem representar estradas, trecho de ruas etc.
- Nessas situações, pode-se associar a cada aresta um peso
  - Para representar a distância entre os vértices.
  - Para representar o tempo que leva para ir de um vértice ao outro.
  - Para representar o custo associado à travessia dela.
- Um problema típico sobre essa situação é definir o comprimento do caminho mais curto entre dois vértices do grafo.

Problema do caminho mínimo

- A distância entre dois vértices será dada pela soma dos pesos das arestas que fazem parte do caminho mínimo entre eles.
  - No caso de grafos não ponderados, o peso de cada aresta será igual a +1.
- Geralmente os problemas de caminho mínimo consideram grafos em que os pesos das arestas sejam todos positivos.
  - Arestas com peso negativo podem representar trajetos em que haja uma remuneração pelo uso dela ao invés de um custo.
  - Arestas com peso negativo podem levar a ocorrência de ciclos infinitos
    - Uso contínuo das arestas de peso negativo diminuem o valor da soma dos pesos e levam a “percepção” de melhora do resultado.

Problema do caminho mínimo

- Observe que:
  - Em um grafo pode haver diversos caminhos que permitem sair do vértice origem e chegar ao vértice destino.
  - Pode não haver nenhum caminho com a origem e o destino desejado.
  - O caminho mínimo corresponde aquele caminho cuja soma dos pesos das arestas é o menor entre todos os caminhos existentes entre o vértice origem e o vértice destino.
  - O caminho mínimo entre dois vértices, se houver, pode ser formado por apenas uma aresta que liga o vértice origem com o vértice destino ou o caminho pode ser composto por um conjunto de arestas e vértices intermediários.
  - Pode haver mais de um caminho com o mesmo custo mínimo.

Problema do caminho mínimo a partir de um vértice

- Considera-se definir os caminhos mínimos entre um vértice origem s e os demais vértices em um grafo G.
- A distância de um vértice v a um vértice u em G é denotada por
  d(v, u)
- O valor da distância é o comprimento do caminho de menor comprimento (ou peso) de v para u, se existir.
  - Se não houver caminho de v para u em G, d(v, u) = $+\infty$
- Para o caso de caminho mínimo em grafos não ponderados, uma opção de solução é dada pelo algoritmo de busca em largura.

Algoritmo Guloso

- Os algoritmos gulosos são denominados assim porque tem como princípio chegar a solução de um problema de otimização realizando a cada passo uma escolha que no momento pareça ser a mais favorável.
- São algoritmos incrementais ou iterativos que avançam optando por uma direção ou escolha baseado em um critério local que atenda melhor ao objetivo geral da otimização solicitada no problema.

Algoritmo Guloso

- No caso do problema do caminho mínimo com origem única, a ideia básica é realizar um caminhamento em largura “ponderado”, iniciando em s.
- A partir de s é formada uma “nuvem” de vértices em torno de s com os vértices passando a fazer parte da nuvem na sequência de suas distâncias de s.
  - O vértice que passará a fazer parte da nuvem é sempre aquele vértice que está mais próximo de s.
- O processo continua até que não existam mais vértices atingíveis a partir de s fora da nuvem.

Algoritmo de Dijkstra

- O algoritmo de Dijkstra para Caminho mínimo com uma única origem em um Grafo G é uma aplicação do Algoritmo Guloso.
- Nem sempre o algoritmo guloso apresenta o melhor resultado para um problema de otimização pois, em sua forma original, ele usa como critério para escolha dos vértices e arestas que farão parte da solução o que tem menor custo, sem rever se a partir da inclusão de um novo vértice, um dos caminhos anteriormente definidos pode ser substituído por outro de custo menor.
- O algoritmo de Dijkstra apresenta uma forma engenhosa de avaliar os custos durante a inclusão de novos vértices, dando a melhor solução para o problema de caminho mínimo.

Algoritmo de Dijkstra

- Define-se um rótulo D[u] para cada vértice do Grafo G, que será usado para aproximar a distância em G do vértice origem s ao vértice u.
  - O rótulo D[u] irá sempre armazenar o comprimento do melhor caminho encontrado até o momento de s até u.
  - D[s] = 0 indica a distância de s para s.
  - D[u] = $+\infty$ no início do algoritmo.
- Define-se um conjunto C que servirá como "nuvem" de vértices para os quais já foram definidas as menores distâncias.
  - C = { } no início do algoritmo

Algoritmo de Dijkstra

- A cada iteração do algoritmo, será selecionado um vértice u que não está em C com o menor rótulo D[u] e colocamos em C.
  - Na primeira iteração, o menor rótulo é encontrado em D[s] = 0.
  - O vértice s é colocado em C.

Algoritmo de Dijkstra

- Quando um novo vértice u é colocado em C, atualizamos o rótulo D[z] de cada vértice z que é adjacente ao vértice u e que ainda não esteja em C, levando em consideração que D[z] deve ser a menor distância conhecida de s a z em uma operação chamada relaxamento:

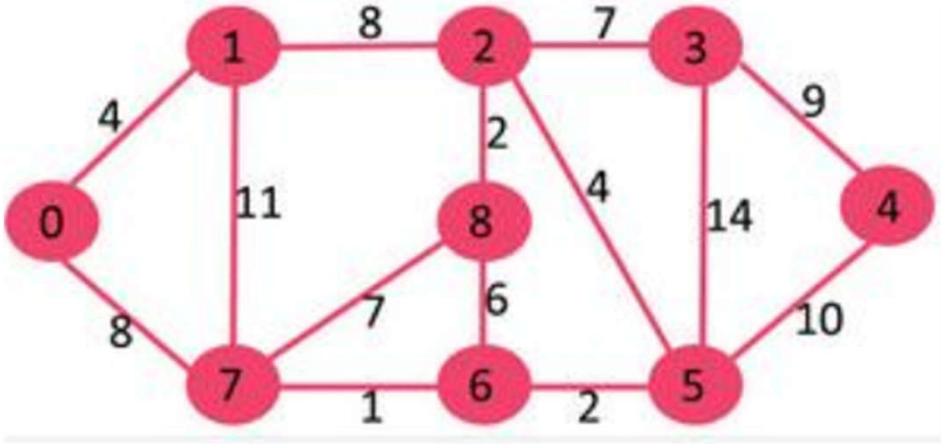
```
Para cada z adjacente a u que não está em C faça
    Se D[u] + peso(u, z) < D[z] então
        D[z] <- D[u] + peso(u, z)
```

- Se o caminho encontrado para z não for melhor do que o caminho antigo, então não mudamos o valor de D[z]

Algoritmo de Dijkstra

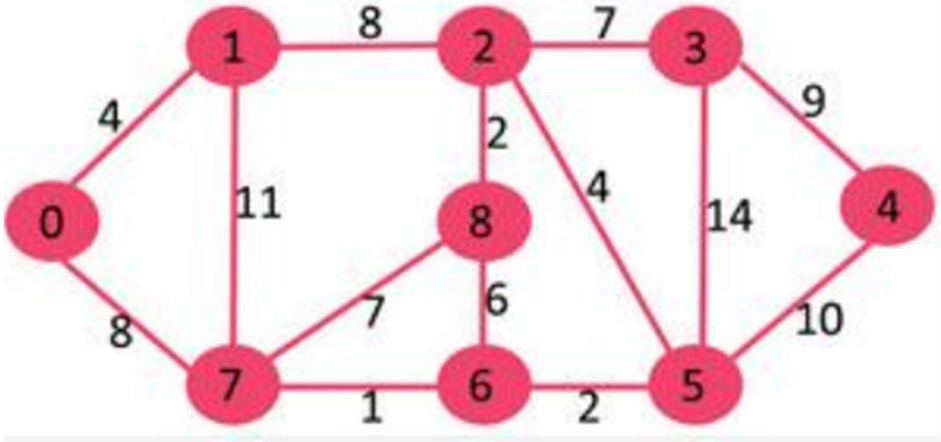
```
Algoritmo Dijkstra(G, s)
  D[s] <- 0
  P[s] <- null //s não tem precedente
  para cada u≠s em G faça D[u] <- +∞  P[u] <- null
   // Q é uma fila de prioridade contendo todos
   // os vértices de G usando D para ordenação da prioridade
  enquanto Q não estiver vazia faça
     u <- Q.removeMin()
     Para cada z adjacente a u que não está em C faça
           Se (D[u] + peso(u, z)) < D[z] então
               D[z] <- D[u] + peso(u, z)
               P[z] <- u // guarda precedente de z
     C <- u;
```

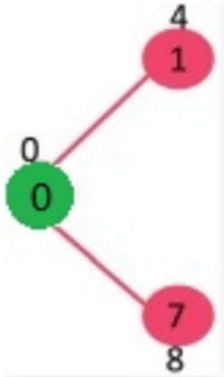
Exemplo



O vértice s = 0
D[s] = 0
P[s] = null
D[*] = +∞

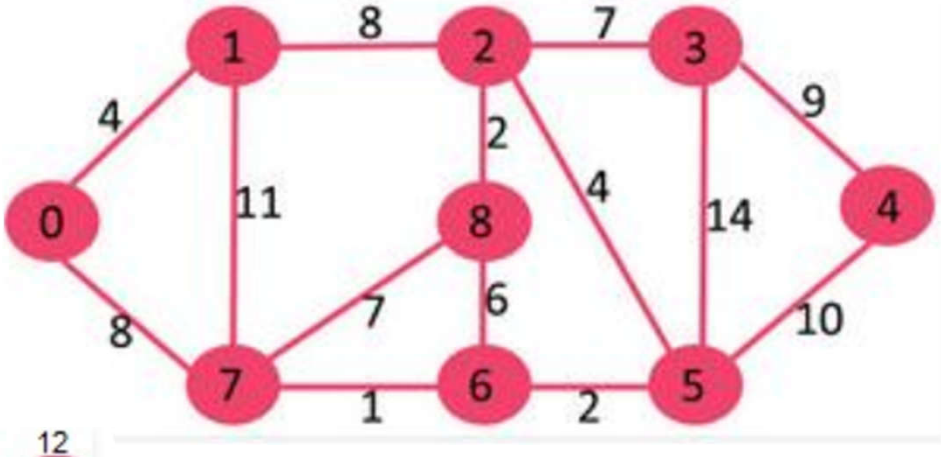
Exemplo

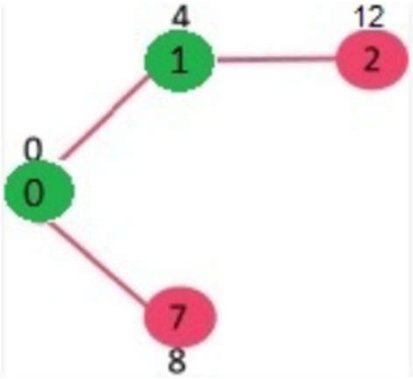




O vértice s = 0
Vértices 1 e 7 são adjacentes a 0
D[1] = 4 e D[7] = 8 D[*] = +∞
P[1] = 0 e P[7] = 0

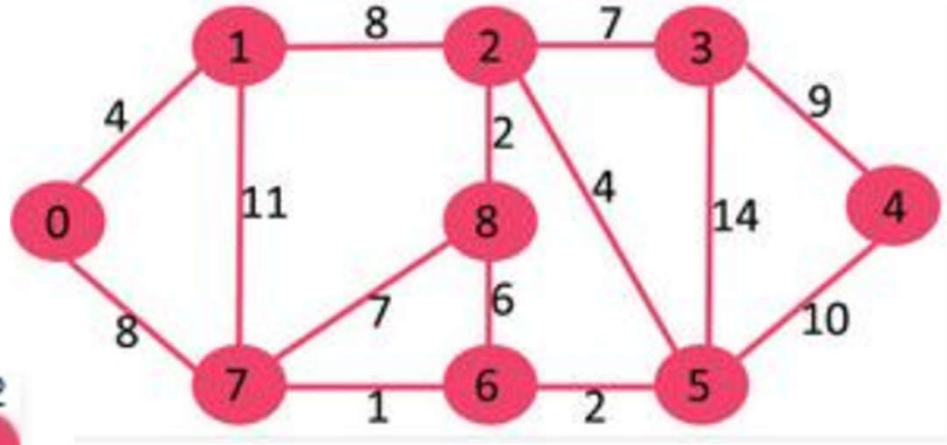
Exemplo

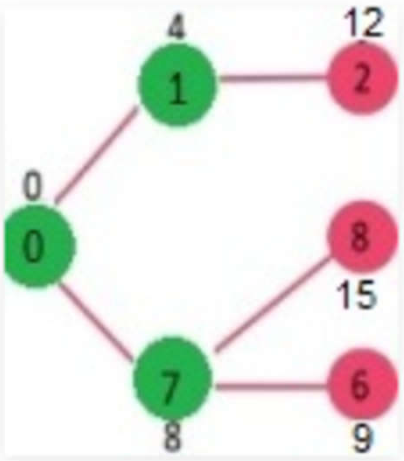




Por ter o menor valor de D[1] = 4, o vértice 1 sai da fila Q e é considerado como u.
Vértices 2 e 7 são adjacentes a 1
D[2] = 12 e D[7] = 8 D[*] = +∞
P[2] = 1 (P[7] não muda pois caminho por 1 é mais longo

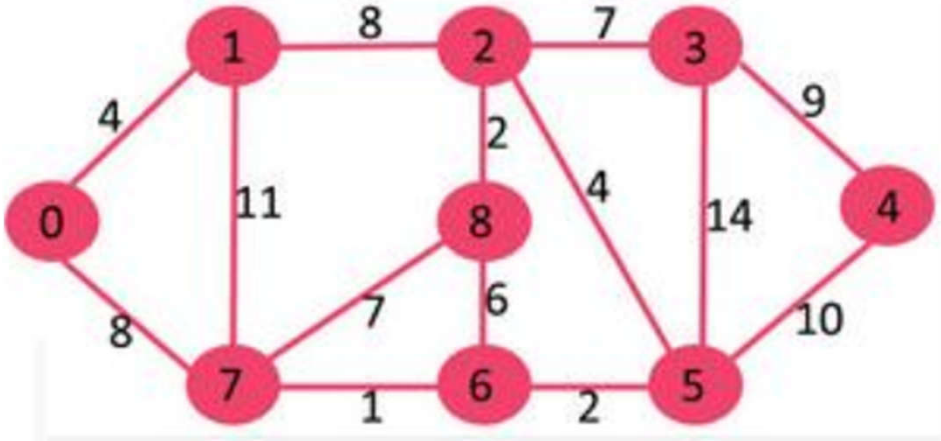
Exemplo

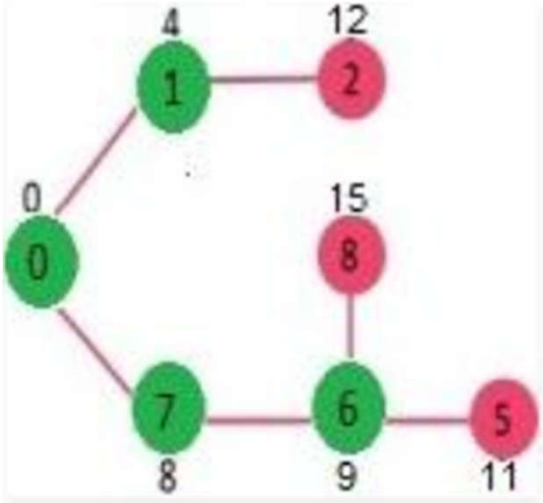




Por ter o menor valor de D[7] = 8, o vértice 7 sai da fila Q e é considerado como u.
Vértices 8 e 6 são adjacentes a 7
D[8] = 15 e D[6] = 9 D[*] = +∞
P[8] = 7 e P[6] = 7

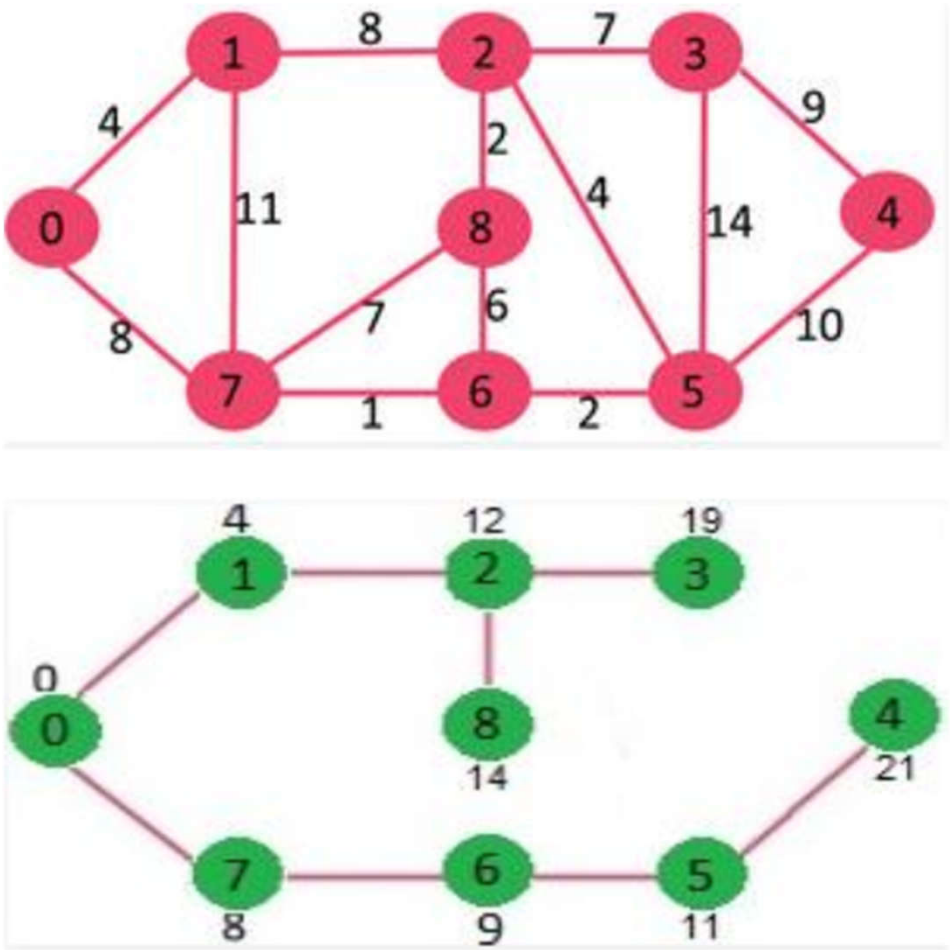
Exemplo





Por ter o menor valor de D[6] = 9, o vértice 9 sai da fila Q e é considerado como u.
Vértices 5 e 8 são adjacentes a 6
D[5] = 15 e D[8] =11 D[*] = +∞
P[5] = 6 e P[8] = 6

Exemplo



Exercício de implementação

- Implementar o algoritmo de Dijkstra para gerar os rótulos D e P sobre as implementações feitas sobre matriz de adjacências e lista de adjacências em um grafo ponderado não direcionado.
- Verificar se o algoritmo implementado consegue resolver o problema do caminho mínimo com uma única origem se o grafo for direcionado.
  - Proponha alterações na implementação, caso sejam necessárias.